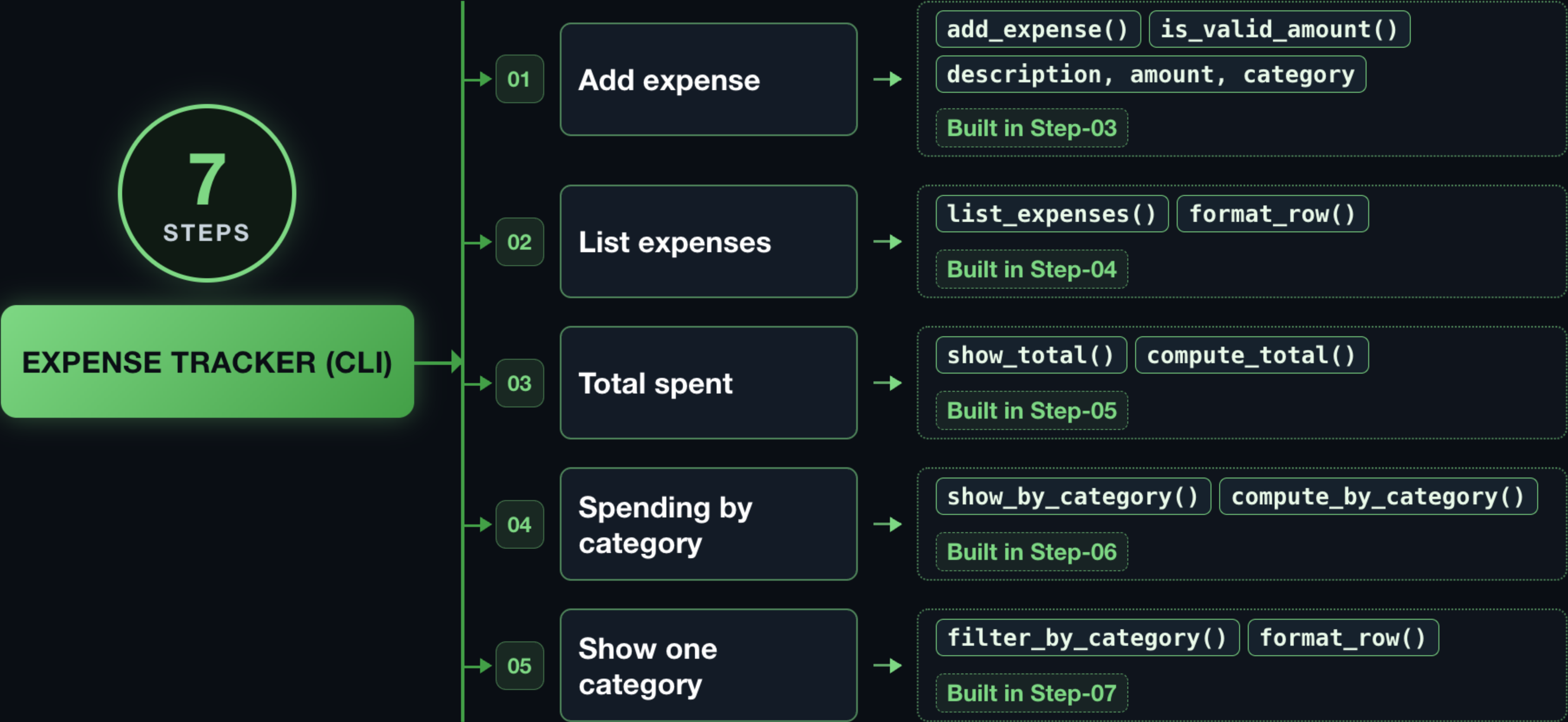


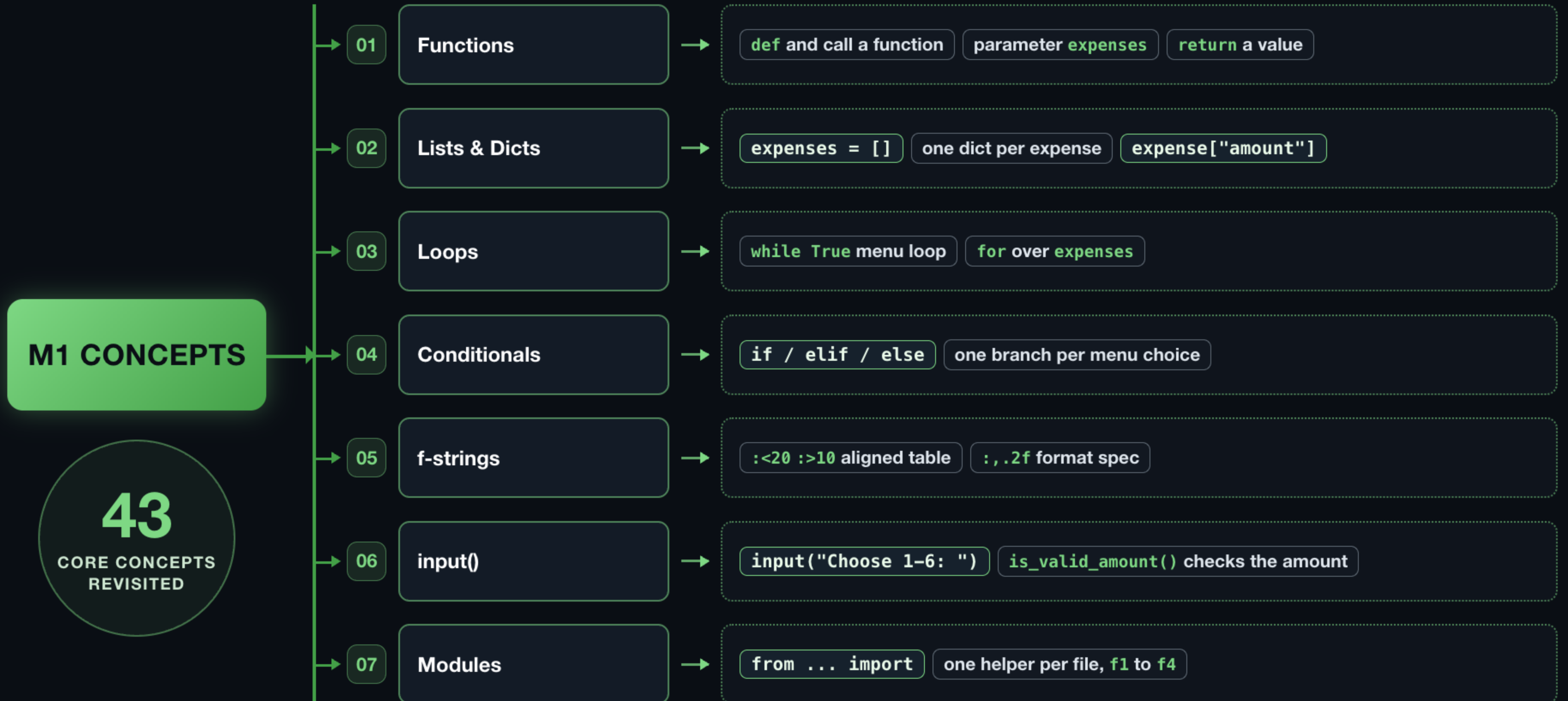
Milestone Project-1: CLI Expense Tracker - What You Will Build?

The five menu actions, and the functions that make each one work.



Project 1: Concept Map

Almost everything in this project you have already learned. This map is the checklist we work through, feature by feature.



Project 1: Which Folder Is Which

Four folders in `milestone-projects/milestone-project-1/`. One of them is yours to type in.

`build-incremental`

THE ANSWER

The finished files, `Steps 01 to 06`. Your answer key.

`build-incremental-lecture`

YOU BUILD HERE

Six empty files. Build `Steps 01 to 06` here.

`solution`

ONE FILE

The whole program in `expense_tracker.py`. Compare at `Step-07`.

`warmup`

OPTIONAL

Six short drills, one per skill. An optional extra.

IN SHORT

Everything you type goes in `build-incremental-lecture`. Compare with `build-incremental` as you go.

Step-01: 1.1 The Menu Shell in `main.py`

`main.py` is the menu driver: it shows the menu, reads a choice, and calls one of five actions.

CONCEPTS USED

Output: `print()`
Section 02

User input: `input()` and `.strip()`
Sections 02, 03

Conditionals: `if / elif / else`
Section 04

Lists: `expenses = []`
Section 05

Loops: `while True` and `break`
Section 06

Functions: `def` and `call`
Section 07

Modules: `from ... import`
Section 07



WHAT WE BUILD

`show_menu()`
prints the banner and six options

`main()`
repeats the menu until you choose 6

`expenses`
the one list every action uses

Choices 1 to 5
call the five actions

Choice 6
prints Goodbye! and stops

Any other input
asks for a number from 1 to 6

Five imported names
the actions from `f5_actions.py`

Step-01: 1.2 Five Stub Actions in `f5_actions.py`

A **stub** is a placeholder that only prints which option you picked and the current list, so the whole menu can run.

CONCEPTS USED

Functions: def and call
Section 07

Parameters: (expenses)
Section 07

Modules: your own .py file
Section 07

Output: print(a, b)
Section 02

Escape sequence: \n
Section 02



WHAT WE BUILD

add_expense()
stub for choice 1

list_expenses()
stub for choice 2

show_total()
stub for choice 3

show_by_category()
stub for choice 4

filter_by_category()
stub for choice 5

Expenses: []
each stub prints the shared list

Step-02: 2.1 Validate the Amount in `f1_is_valid_amount.py`

`is_valid_amount()` checks the typed text with string methods first, then `float()`: True for a positive number, False for anything else.

CONCEPTS USED

String methods: `.strip()`, `.split()`
Sections 03, 05

String method: `.isdecimal()`
New in this project

Lists: `len()` and `[0]`
Section 05

Conditionals: `if / elif / else`
Section 04

Logical operators: `and`, `not`
Section 04

Type conversion: `float()`
Section 02

Return values: `return`
Section 07



WHAT WE BUILD

`is_valid_amount()`
returns True or False

Spaces trimmed
" 12 " is read as 12

Whole number
12 passes

One dot
12.50 passes

Not digits
-5, abc and 1.2.3 fail

Not above zero
0 and 0.00 fail

Safe `float()`
runs only after the digit check

Step-02: 2.2 Wire Up the Add Action in `f5_actions.py`

`add_expense()` asks for three values, checks the amount with `is_valid_amount()`, and adds one expense to the list.

CONCEPTS USED

Modules: `from ... import`
Section 07

User input: `input()`
Section 02

String methods: `.strip()`, `.lower()`
Section 03

Conditionals: `if`, `if not`
Section 04

Early return: `return`
Section 07

Type conversion: `float()`
Section 02

Dictionaries and lists: `{...}`, `.append()`
Section 05



WHAT WE BUILD

`add_expense()`
the real action for choice 1

The import
`is_valid_amount` from `f1`

Empty description
becomes (no description)

Bad amount
prints a message, nothing is added

Category
lowercased; empty becomes uncategorized

One expense
a dict, appended to expenses

Choice 1 output
Added: Coffee (3.50) [food]

Step-03: 3.1 One Aligned Row in `f2_format_row.py`

`format_row()` turns one expense into one line of text, every column the same width.

CONCEPTS USED

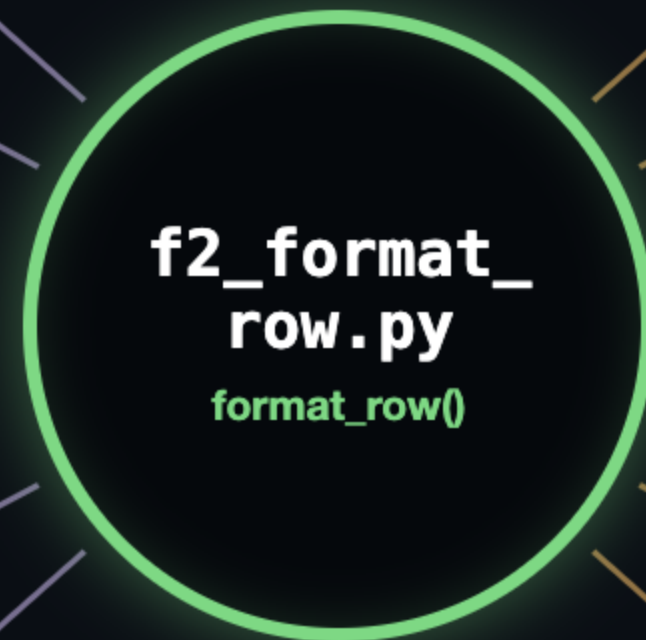
Dictionary lookup: `expense["key"]`
Section 05

f-strings: `f"{value}"`
Section 03

Width and alignment: `:<20 :>10`
New in this project

Format spec: `:', .2f`
Section 03

Return values: `return`
Section 07



WHAT WE BUILD

`format_row()`
returns one aligned row

Three columns
description, category, amount

Column widths
20, 14 and 10 characters

Amount
right-aligned, two decimals

A pure helper
prints nothing by itself

Step-03: 3.2 Wire Up the List Action in f5_actions.py

`list_expenses()` prints a header, then one `format_row()` line for each expense.

CONCEPTS USED

Modules: `from ... import`
Section 07

Truthy and falsy: `if not`
Section 04

Early return: `return`
Section 07

Loops: `for` over a list
Section 06

Width and alignment: `:<20 :>10`
New in this project

WHAT WE BUILD

`list_expenses()`
the real action for choice 2

The import
`format_row` from `f2`

Empty list
prints No expenses yet.

Header row
the same widths as the rows

One row per expense
`format_row(expense)`

f5_actions.py
`list_expenses()`

Step-04: 4.1 Add Every Amount in f3_compute_total.py

`compute_total()` adds every amount in the list and returns one number.

CONCEPTS USED

Data types: float
Section 02

Loops: for over a list
Section 06

Dictionary lookup: `expense["key"]`
Section 05

Augmented assignment: `+=`
Section 04

Accumulator: a running total
Section 06

Return values: `return`
Section 07



WHAT WE BUILD

`compute_total()`
returns the sum of all amounts

`total`
starts at 0.0

Each expense
adds its amount with `+=`

Empty list
returns 0.0

A pure helper
prints nothing by itself

Step-04: 4.2 Wire Up the Total Action in f5_actions.py

`show_total()` calls `compute_total()` and prints the result as money.

CONCEPTS USED

Modules: `from ... import`
Section 07

Functions: `def` and `call`
Section 07

f-strings: `f"{value}"`
Section 03

Format spec: `:.2f`
Section 03

WHAT WE BUILD

`show_total()`
the real action for choice 3

The import
`compute_total` from `f3`

No expenses
prints Total spent: 0.00

Choice 3 output
Total spent: 5.50

f5_actions.py
`show_total()`

Step-05: 5.1 Totals per Category in `f4_compute_by_category.py`

`compute_by_category()` keeps one running total per category and returns them in a dictionary.

CONCEPTS USED

Dictionaries: totals = {}
Section 05

Loops: for over a list
Section 06

Dictionary lookup: expense["key"]
Section 05

Dictionary method: .get() with a default
Section 05

Accumulator: a running total
Section 06

Return values: return
Section 07

f4_compute_by_category.py
`compute_by_category()`

WHAT WE BUILD

compute_by_category()
returns a dict: category to total

New category
starts at 0.0, no KeyError

Same category
adds to its own total

Empty list
returns an empty dict

A pure helper
prints nothing by itself

Step-05: 5.2 Wire Up the Category Action in `f5_actions.py`

`show_by_category()` asks `compute_by_category()` for the totals and prints one line per category.

CONCEPTS USED

Modules: `from ... import`
Section 07

Truthy and falsy: `if not`
Section 04

Early return: `return`
Section 07

Loops: `for with .items()`
Sections 05, 06

Width and alignment: `:<14 :>10`
New in this project

WHAT WE BUILD

`show_by_category()`
the real action for choice 4

The import
`compute_by_category` from `f4`

Empty dict
prints No expenses yet.

One line per category
category, then its total

Choice 4 output
food 15.50 and transport 2.00

`f5_actions.py`
`show_by_category()`

Step-06: 6.1 The Last Action in `f5_actions.py`

`filter_by_category()` asks for a category and shows only the expenses in it.

CONCEPTS USED

User input: `input().strip().lower()`
Sections 02, 03

List comprehension: filter with `if`
Section 06

Truthy and falsy: `if not`
Section 04

Early return: `return`
Section 07

f-strings: `f"{value}"`
Section 03

Width and alignment: `:<20 :>10`
New in this project



WHAT WE BUILD

`filter_by_category()`
the real action for choice 5

`matches`
a new list: only the typed category

No matches
prints No expenses in 'travel'.

Matches
a header, then `format_row()` rows

Step-06: 6.2 The Finished `f5_actions.py`

The finished `f5_actions.py`, top to bottom: four imports, then the five actions the menu calls.

CONCEPTS USED

Modules: `from ... import`
Section 07

Functions: `def` and `call`
Section 07

Parameters: `(expenses)`
Section 07

`f5_actions.py`
all five actions

WHAT WE BUILD

Four imports
f1 to f4 at the top of the file

Five actions
add_expense to filter_by_category

Every action
takes the same expenses list

Helpers
return a value and print nothing

Three layers
f1-f4, `f5_actions.py`, `main.py`

Step-07: Run It All and Compare `expense_tracker.py`

Run every menu choice in your build, then read the same program in **ONE** file: same inputs, same output.

CONCEPTS USED

Run a program: `python3`
Section 02

Functions: `def` and `call`
Section 07

Modules: `from ... import`
Section 07

Entry point: `main()`
New in this project



WHAT WE BUILD

Choices 1 to 6
all work in one run of `main.py`

Six files
`f1` to `f4`, `f5_actions`, `main`

`expense_tracker.py`
all 11 functions in one file

No imports
nothing imported between files

Same output
Total spent: 5.50 from both files